ASE CHAPTER 2

T5 Apply & Scenario Selection Pack · Life-Cycle Models

Applied Decision Analysis and Justified Model Selection for Real-World Systems

This pack contains the core **applied scenario selection and decision-justification exercises** from printed page 52 of the primary course textbook (B.B. Agarwal et al., 2010, Exercise 13, covering four distinct software engineering applications), integrated with the four formal selection criteria matrices from K.K. Aggarwal & Yogesh Singh (2007, Slides 36–41).

> **Formatting Standard:** - Questions and justified model recommendations are strictly in English. - **Bold marks source-grounded model selection rules, criteria, and characteristics**; plain text marks structural application analysis. - No markdown tables are used in question/answer bodies (pure text formatting preserves question-answer cohesion). - Detailed Arabic notes and exam traps are placed separately outside the English model answers.

Exercise 13 (Overview): Engineering Principles of Life-Cycle Model Selection

Selecting an SDLC model is an engineering decision governed by evaluating four interrelated project parameters rather than universal preference: 1. **Requirements Certainty:** Are requirements fully understood and fixed upfront, or fluid, evolving, and unclear? 2. **Technical and Project Risk:** Does the project rely on unproven technologies, tight hardware constraints, or complex distributed integrations? 3. **User Involvement Profile:** Is the customer available for continuous daily feedback, or restricted to formal milestone approvals? 4. **Development Team and Delivery Constraints:** Does the team possess deep domain expertise, and are delivery deadlines aggressive (60–90 days) requiring parallel modular development?

Exercise 13.i: Which life-cycle model would you follow for developing software for: A Game? Justify your selection with an appropriate reason.

Recommended Model: Prototyping Model (or Evolutionary Prototyping / Agile)

Engineering Justification: - **Extreme User Experience and Interface Uncertainty:** Video games depend heavily on subjective human factors—gameplay mechanics, visual appeal, user immersion, graphical responsiveness, and intuitive controls—which cannot be captured in a static upfront SRS document. - **Exploratory Playtesting and Refinement: The prototyping paradigm enables rapid development of working gameplay mock-ups.** These prototypes are evaluated directly through user playtesting to tune controls, adjust difficulty pacing, and eliminate disorienting interface flaws before committing to full-scale graphic asset production and rendering engine architecture. - **Technical Risk Resolution:** Allows developers to test 3D rendering framerates, physics calculations, and hardware compatibility on target consoles before locking down the design.

> **ملاحظة الطالب والفخ الامتحاني:** - ألعاب الفيديو تعتمد بالكامل على تجربة المستخدم (Gameplay & UX & GUI)؛ لا يمكن كتابة مواصفات جامدة لها مسبقاً في الشلال. البروتوتايب هو الخيار الهندسي الأمثل لاختبار متعة اللعبة وضبط تفاعلها.

Sources: B.B. Agarwal et al. (2010), §2.3, pp. 41–44; §2.8, p. 51 (Table 2.2); K.K. Aggarwal & Singh (2007) Slides 37–40.

Exercise 13.ii: Which life-cycle model would you follow for developing software for: A New Text Editor? Justify your selection with an appropriate reason.

Recommended Model: Iterative Enhancement Model

Engineering Justification: - **Clear, Stratifiable Core Functionality:** Text editors possess a well-understood, universally recognized functional core: basic character input, file buffer loading, saving, cut-copy-paste, and document printing. - **Incremental Expansion of Capability: The iterative enhancement model is ideally suited because software can be partitioned into distinct increments delivered sequentially to users (matching the canonical example in B.B. Agarwal et al., pp. 47–48):** - **Increment 1 (Core Product): Basic file management, editing, and document production functions.** - **Increment 2: More sophisticated editing and document production capabilities.** - **Increment 3: Spelling and grammar checking capabilities.** - **Increment 4: Advanced page layout capability.** - **Immediate Utility and Feedback:** Users obtain a functional, lightweight core product early on, while subsequent releases enhance advanced capabilities based on actual operational feedback.

> **ملاحظة الطالب والفخ الامتحاني:** - محرر النصوص هو المثال الحرفي الذي وضعه كتاب أغروال ص.47–48 لشرح Iterative Enhancement Model. احرص على ذكر النواة الأساسية أولاً ثم الإضافات التدريجية في كل إصدار.

Sources: B.B. Agarwal et al. (2010), §2.6, pp. 47–48; §2.8, p. 51 (Table 2.2).

Exercise 13.iii: Which life-cycle model would you follow for developing software for: A Compiler for a New Language? Justify your selection with an appropriate reason.

Recommended Model: Waterfall Model (Linear Sequential Model) — Conditional Recommendation

Engineering Justification & Explicit Assumptions: - **Contextual Assumption (Stable Language Definition):** The exercise statement does not specify whether the language specification is stable or evolving. Assuming the formal grammar, lexical syntax, semantics, and target instruction set are established and stable upfront: - **Alignment with Selection Matrices:** According to the Requirements Characteristics selection matrix (K.K. Aggarwal & Singh, Slide 37), the Waterfall model is recommended when **"requirements are easily understandable and defined early"** and **"can we define requirements early in the cycle: Yes"**. - **Sequential Architectural Pipeline:** Classic compiler architecture follows an unbranching pipeline of successive phases: **Lexical Analysis (Scanning)** → **Syntax Analysis (Parsing)** → **Semantic Analysis** → **Intermediate Code Generation** → **Code Optimization** → **Code Generation.** - **Alternative Condition (Experimental / Evolving Language):** If the programming language grammar or type system is experimental and evolving, the Waterfall model is unsuitable because changes to syntax invalidate parser tables and AST definitions across downstream passes; an Evolutionary or Incremental development approach would be required instead.

> **ملاحظة الطالب والفخ الامتحاني:** - نص السؤال لم يحدد حالة المتطلبات؛ لذلك اجعل إجابتك في الامتحان مشروطة بافتراض هندسي صريح: - إذا افترضنا أن مواصفات وقواعد اللغة مستقرة ومفهومة مسبقاً ← نموذج الشلال (Waterfall) خيار قياسي ومعقول ومطابق لمصفوفة الاختيار (Slide 37) نظراً لخطية مراحل بناء المترجم وغياب غموض الواجهات. - إذا كانت اللغة تجريبية وقواعدها تتغير باستمرار ← يفشل الشلال ويتطلب الأمر نموذجاً تطورياً أو تكرارياً.

Sources: B.B. Agarwal et al. (2010), §2.2, pp. 36–41; §2.8, p. 51 (Table 2.2); K.K. Aggarwal & Singh (2007) Slide 37.

Exercise 13.iv: Which life-cycle model would you follow for developing software for: A Software for Hospital Management? Justify your selection with an appropriate reason.

Recommended Model: Spiral Model (or Iterative Enhancement / RAD under explicit conditions)

Contextual Assumptions and Engineering Justification: - The problem statement does not provide detailed operational parameters; therefore, the recommended model depends on the project's risk profile, scale, and integration complexity. - **Primary Recommendation (Spiral Model — Complex / High-Risk Assumption):** - **If the hospital management system is large-scale, unprecedented for the organization, or encompasses mission-critical clinical workflows** (such as real-time patient monitoring, pharmaceutical management, and multi-department database synchronization across multiple locations): - **The Spiral Model is best suited because it is explicitly risk-driven. Alternatives are evaluated based on risk perceptions in Quadrant 2, and targeted strategies (such as prototyping, simulation, and benchmarking) are deployed in Quadrant 3 to resolve complex technical, operational, and clinical risks before large-scale system construction.** - **Viable Alternative Recommendation (Iterative Enhancement — Phased Business Delivery Assumption):** - **If the hospital requires routine administrative automation** (such as patient admissions, room allocation, and billing) where requirements are generally understood but resources are phased: - **The Iterative Enhancement model is appropriate because an administrative core product can be delivered first**, followed by subsequent increments for pharmacy, laboratory, and inventory management based on operational feedback. - **Viable Alternative Recommendation (RAD Model — Modular / Time-Critical Assumption):** - **If the system can be cleanly partitioned into independent modular subsystems** (e.g., front-desk, pharmacy, billing) to be built by multiple experienced teams using pre-existing components with active staff participation within 60–90 days.

> **ملاحظة الطالب والفخ الامتحاني:** - نص السؤال لم يحدد تفاصيل المستشفى، لذلك التبرير يعتمد على الافتراض الهندسي: - إذا افترضنا نظاماً ضخماً عالي المخاطر ومتعدد الأقسام ← النموذج الحلزوني (Spiral Model) لإدارة المخاطر. - إذا افترضنا أتمتة إدارية روتينية تطلق نواة أساسية ثم تتوسع تدريجياً ← نموذج التحسين التكراري (Iterative Enhancement). - إذا كان النظام قابلاً للتقسيم إلى موديولات مستقلة وفرق متعددة ← نموذج RAD. اذكر الافتراض بوضوح في جواب الامتحان.

Sources: B.B. Agarwal et al. (2010), §2.4, pp. 44–46; §2.6, pp. 47–48; §2.7, pp. 49–50; §2.8, pp. 50–51 (Table 2.2); K.K. Aggarwal & Singh (2007) Slides 37–40.

Decision Matrix Summary: The Four Selection Pillars

(Cross-Reference from K.K. Aggarwal & Yogesh Singh, Slides 37–40)

When justifying model selection in an examination, always ground the decision in one or more of the four authoritative source matrices:

Characteristics of Requirements:
Understandable and defined early → **Waterfall / RAD**
Changing frequently → **Prototyping / Spiral**
High complexity → **Prototyping / Iterative / Evolutionary / Spiral**

Status of Development Team:

Less experience on similar projects → **Prototyping / Spiral**
New to technology → **Waterfall / Iterative / Evolutionary / Spiral**
Unfamiliar with development tools → **Waterfall / Spiral**

User Involvement:

Active involvement across all phases → **Prototyping / RAD**
Limited user participation available → **Waterfall / Iterative / Evolutionary / Spiral**

Project Type and Associated Risk:

Enhancement of existing system → **Iterative Enhancement / Evolutionary / RAD**
High reliability requirements → **Iterative / Evolutionary / Spiral**
Tight project schedule → **RAD / Iterative / Evolutionary**
Severe resource scarcity → **Prototyping / Spiral**

> **ملاحظة الطالب والفخ الامتحاني:** - إذا طُلب منك تبرير اختيار نموذج في الامتحان، لا تكتفِ بذكر اسم النموذج، بل اذكر المعيار (المتطلبات، المخاطرة، الفريق، مشاركة المستخدم) واستشهد بالصفات المطابقة له.

Sources: K.K. Aggarwal & Singh (2007) Slides 37–40; B.B. Agarwal et al. (2010), §2.8, pp. 50–51.

Retrieval set — Focused Active Recall Drill

1 **Under what explicit condition does the Waterfall model fail for compiler development, and which model replaces it?**

The Waterfall model fails if the programming language grammar, syntax, and type system are experimental or evolving during development. Mid-stream modifications to grammar break downstream parser tables and AST generators across the pipeline, requiring an Evolutionary or Incremental development approach to evaluate syntax iteratively.

Source: K.K. Aggarwal & Singh (2007) Slide 37; B.B. Agarwal et al. (2010), §2.2, p. 38.

2 **Why does the RAD model fail when applied to pioneering or unproven technologies?**

Because RAD relies on accelerated 60–90 day delivery cycles using established high-level visual tools, automated code generators, and well-understood reusable components. Unproven technologies introduce high technical risk, performance uncertainties, and integration hurdles that cannot be accommodated within rigid, compressed timeboxes.

Source: K.K. Aggarwal & Singh (2007) Slide 12, Slide 38; B.B. Agarwal et al. (2010), §2.7, p. 50.